MODULE 3

Transaction Management and Concurrency Control

What is a Transaction?

- Collections of operations that form a single logical unit of work are called **transactions**.
- A database system must ensure proper execution of transactions despite failures—either the entire transaction executes, or none of it does.
- **Transaction** is a unit of program execution that accesses and possibly updates various data items.

In database terms,

- A transaction is any action that reads from or writes to a database. A transaction may consist of the following:
  - A simple SELECT statement to generate a list of table contents.
  - A series of related UPDATE statements to change the values of attributes in various tables.
  - A series of INSERT statements to add rows to one or more tables.
  - A combination of SELECT, UPDATE, and INSERT statements.
- Example: The sales transaction example includes a combination of INSERT and UPDATE statements.

- A *logical* unit of work that must be either entirely completed or aborted
- Successful transaction changes the database from one *consistent* state to another
  - One in which all data integrity constraints are satisfied
- Most real-world database transactions are formed by two or more database requests
  - The equivalent of a single SQL statement in an application program or transaction

Evaluating Transaction Results

Evaluating Transaction Results

- Not all transactions update the database. Suppose that you want to examine the CUSTOMER table to determine the current balance for customer number 10016. Such a transaction can be completed by using the following SQL code:
- SELECT CUST_NUMBER, CUST_BALANCE FROM CUSTOMER WHERE CUST_NUMBER = 10016;
- Although the query does not make any changes in the CUSTOMER table, the SQL code represents a transaction because it accesses the database.
- If the database existed in a consistent state before the access, the database remains in a consistent state after the access because the transaction did not alter the database.

TRANSACTION CONCEPT

- A transaction is a unit of program execution that accesses and possibly updates various data items.
- A transaction is made to change data in a database which can be done by inserting new data, updating the existing data, or by deleting the data that is no longer required.
- There are certain types of transaction states which tell the user about the current condition of that database transaction and what further steps to be followed for the processing.
- Read(X): This read operation is applied to read the X's value from the database server and keeps it in a buffer in the main memory.
- Write(X): This write operation is applied to write the X's value back to the database server from the buffer.

Example: transaction to transfer $50 from account A to account B:

1. read(A)
2. A := A – 50
3. write(A)
4. read(B)
5. B := B + 50
6. write(B)

➢ Two main issues to deal with:

- Failures of various kinds, such as hardware failures and system crashes - Recovery
- Concurrent execution of multiple transactions

Transaction Properties(**ACID** properties)

About ACID Properties

- It is important to ensure that the database remains consistent before and after the transaction.
- To ensure the consistency of database, certain properties are followed by all the transactions occurring in the system.
- These properties are called as ACID Properties of a transaction.

Atomicity

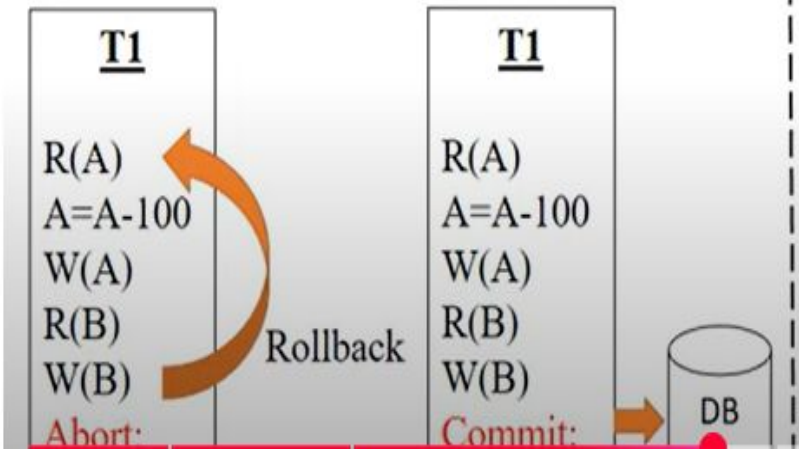
- Requires that ***all*** operations (SQL requests) of a transaction be completed; if not, then the transaction is aborted
- A transaction is treated as a single, indivisible, logical unit of work
- This **"all-or-none"** property is referred to as atomicity.
- Example: If a transaction T1 has four SQL requests, all four requests must be successfully completed; otherwise, the entire transaction is aborted.
- In other words, a transaction is treated as a single, indivisible, logical unit of work.

1. Atomicity

➢ **(ALL or NONE)**

- This property ensures that either the transaction occurs completely or it does not occur at all.
- In other words, it ensures that no transaction occurs partially.
- It is the responsibility of Transaction Control Manager to ensure atomicity of the transactions.
- The operations either get Abort or Commit.

Examples:



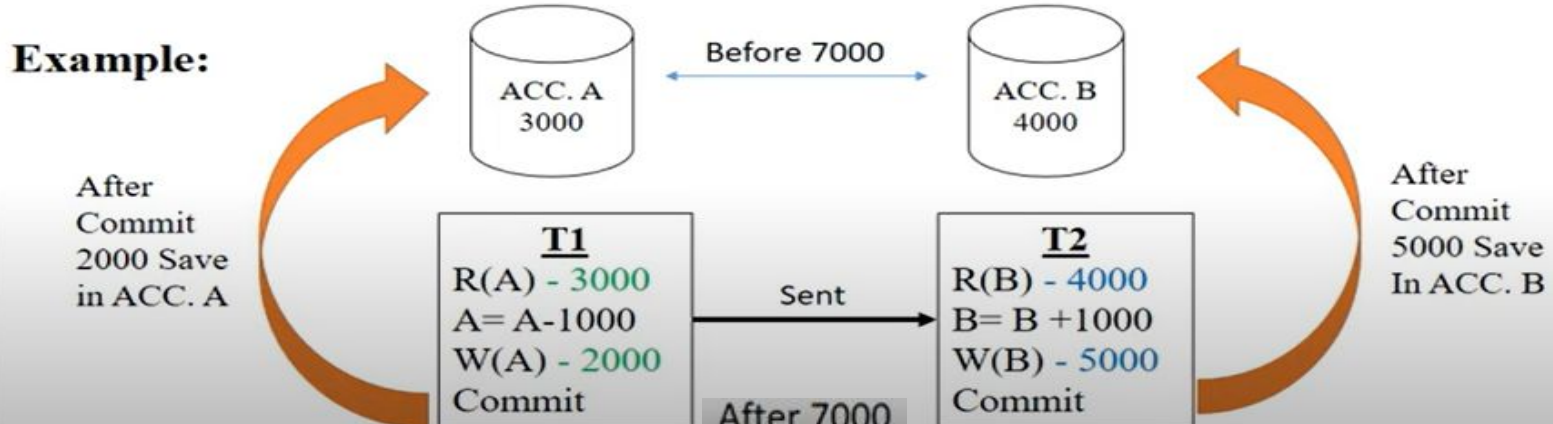
Consistency

- Consistency property ensures that the database must remain in **the consistent state before the start of transaction and after the transaction is over.**
- Consistency states that only valid data will be written to the database.
- If for some reason a transaction is executed that violates the database consistency rules the entire transaction will be rolled back.

➢ **(Before & After the Transection, Sum same)**

- This property ensures that integrity constraints are maintained.
- It ensures that the database remains consistent before and after the transaction.
- It is the responsibility of DBMS application programmer to ensure consistency of the database.

Example:



Isolation(This occurs before commit statement)

- Data used during execution of a transaction cannot be used by second transaction until first one is completed
- Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions Ti and Tj , it appears to Ti that either Tj finished execution before Ti started, or Tj started execution after Ti finished.
- In other words, if transaction
- T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2 … Tn) until T1 ends. This property is particularly useful in multiuser database environments because several users can access and update the database at the same time.

- **Durability**

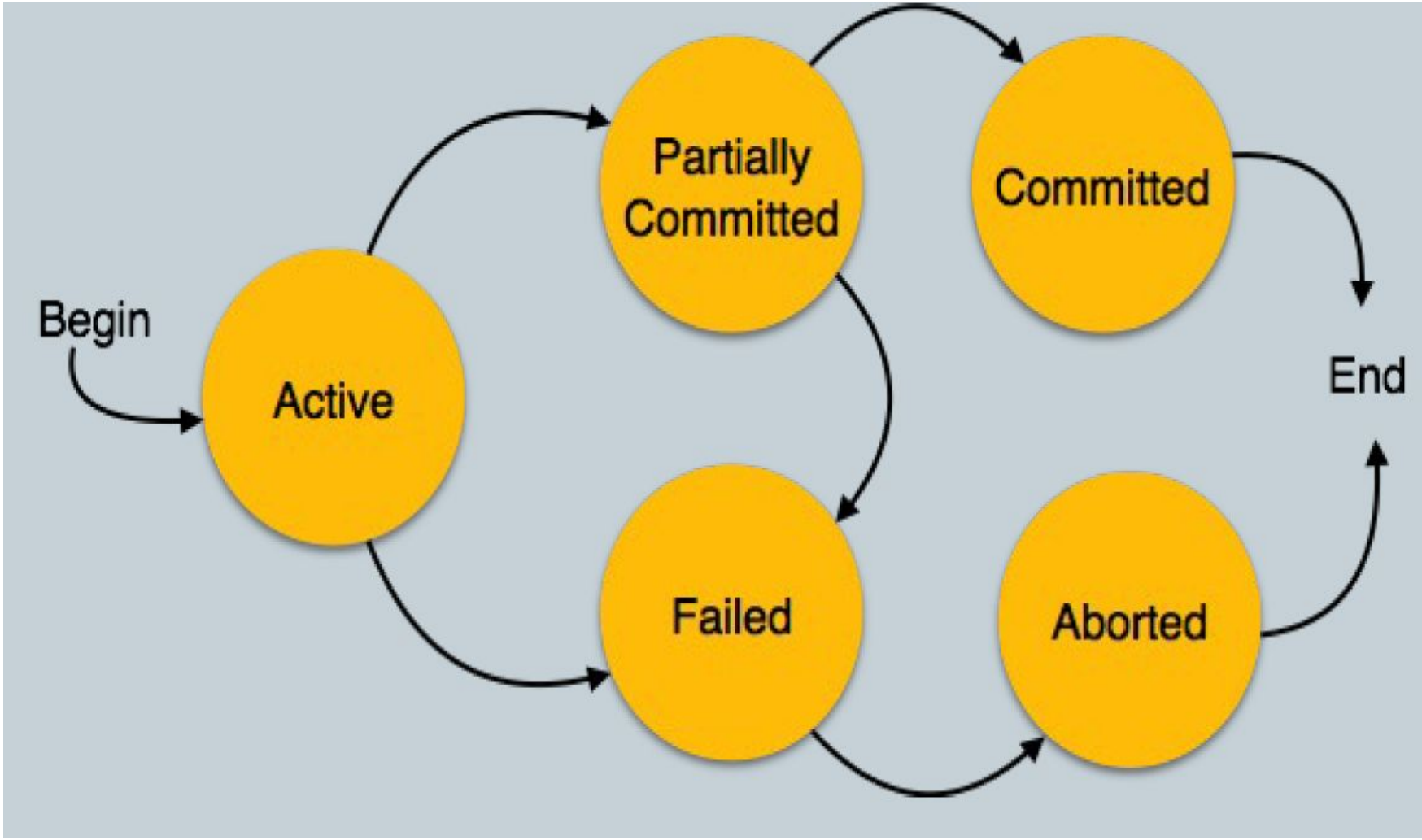
Durability ensures that once transaction changes are done and committed, they cannot be undone or lost, even in the event of a system failure.

- After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.
- Durability can be implemented by writing all transaction into a **transaction log** that can be used to create a system state right before failure.
- A transaction can only regard as committed after it is written safely in the log.
- For example, in an application that transfers funds from one account to another, the durability property ensures that the changes made to each account will not be reversed.
- These properties are called the **ACID properties.**

Transaction State

A transaction must be in one of the following states:

- **Active:-**
  - The initial state; the transaction stays in this state while it is executing.
- **Partially committed:-**
  - After the final statement has been executed.
- **Failed:-**
  - After the discovery that normal execution can no longer proceed.
- **Aborted:-**
  - After the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction
- **Committed:-**
  - After successful completion


Begin
Active
Partially Committed
Committed
Failed
Aborted
End

Transaction Management with SQL

The American National Standards Institute (ANSI) has defined standards that govern SQL database transactions. Transaction support is provided by two SQL statements: COMMIT and ROLLBACK. The ANSI standards require that when a transaction sequence is initiated by a user or an application program, the sequence must continue through all succeeding SQL statements until one of the following four events occurs:

- A COMMIT statement is reached, in which case all changes are permanently recorded within the database. The COMMIT statement automatically ends the SQL transaction.
- A ROLLBACK statement is reached, in which case all changes are aborted and the database is rolled back to its previous consistent state.
- The end of a program is successfully reached, in which case all changes are permanently recorded within the database. This action is equivalent to COMMIT.
- The program is abnormally terminated, in which case the database changes are aborted and the database is rolled back to its previous consistent state. This action is equivalent to ROLLBACK.

- The use of COMMIT is illustrated in the following simplified sales example, which updates a product's quantity on hand (PROD_QOH) and the customer's balance when the customer buys two units of product 1558-QW1 priced at $43.99 per unit (for a total of $87.98) and charges the purchase to the customer's account:

```
UPDATE  PRODUCT
SET PROD_QOH = PROD_QOH – 2
WHERE PROD_CODE = '1558-QW1';

UPDATE CUSTOMER
SET CUST_BALANCE = CUST_BALANCE + 87.98
WHERE CUST_NUMBER = '10011';
COMMIT;
```

The Transaction Log

A DBMS uses a **transaction log** to keep track of all transactions that update the database. The DBMS uses the information stored in this log for a recovery requirement triggered by a ROLLBACK statement, a program's abnormal termination, or a system failure such as a network discrepancy or a disk crash. Some RDBMSs use the transaction log to recover a database *forward* to a currently consistent state. After a server failure, for example, Oracle automatically rolls back uncommitted transactions and rolls forward transactions that were committed but not yet written to the physical database. This behavior is required for transactional correctness and is typical of any transactional DBMS.

While the DBMS executes transactions that modify the database, it also automatically updates the transaction log. The transaction log stores the following:

The Transaction Log

- Keeps track of all transactions that update the database. It contains:
  - A record for the beginning of transaction
  - For each transaction component (SQL statement)
    - Type of operation being performed (update, delete, insert)
    - Names of objects affected by the transaction (the name of the table)
    - “Before” and “after” values for updated fields
    - Pointers to previous and next transaction log entries for the same transaction
  - The ending (COMMIT) of the transaction
- Increases processing overhead but the ability to restore a corrupted database is worth the price

- Increases processing overhead but the ability to restore a corrupted database is worth the price
- If a system failure occurs, the DBMS will examine the log for all uncommitted or incomplete transactions and it will restore the database to a previous state
- The log it itself a database and to maintain its integrity many DBMSs will implement it on several different disks to reduce the risk of system failure

TABLE 9.1 A TRANSACTION LOG

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				

TRL_ID = Transaction log record ID

PTR = Pointer to a transaction log record ID

TRX_NUM = Transaction number

(Note: The transaction number is automatically assigned by the DBMS.)

CONCURRENCY CONTROL

- Coordinating the simultaneous execution of transactions in a multiuser database system is known as concurrency control.
- The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment.
- To achieve this goal, most concurrency control techniques are oriented toward preserving the isolation property of concurrently executing transactions.
- Concurrency control is important because the simultaneous execution of transactions over a shared database can create several data integrity and consistency problems.
- The three main problems are lost updates, uncommitted data, and inconsistent retrievals.
    1. **Lost Updates**
    2. **Uncommitted Data**
    3. **Inconsistent Retrievals**

Lost Updates

- The lost update problem occurs when two concurrent transactions, T1 and T2, are updating the same data element and one of the updates is lost (overwritten by the other transaction).
- To illustration of lost updates, examine a simple PRODUCT table. One of the table's attributes is a product's quantity on hand (PROD_QOH).
- Assume that you have a product whose current PROD_QOH value is 35.
- Also assume that two concurrent transactions, T1 and T2, occur and update the
- PROD_QOH value for some item in the PRODUCT table.
- The transactions are shown in Table 10.2.

TABLE 10.2

TWO CONCURRENT TRANSACTIONS TO UPDATE QOH

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100
T2: Sell 30 units	PROD_QOH = PROD_QOH – 30

TABLE 10.3

SERIAL EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	PROD_QOH = 135 – 30	
6	T2	Write PROD_QOH	105

Table 10.3 shows the serial execution of the transactions under normal circumstances, yielding the correct answer PROD_QOH = 105.

- suppose that a transaction can read a product's PROD_QOH value from the table before a previous transaction has been committed, using the same product.
- The sequence depicted in Table 10.4 shows how the lost update problem can arise.
- Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed.
- Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory.
- In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is "lost" during the process.

TABLE 10.4

LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	PROD_QOH = 35 + 100	
4	T2	PROD_QOH = 35 – 30	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted Data

- The phenomenon of uncommitted data occurs when two transactions, T1 and T2, are executed concurrently and the first transaction (T1) is rolled back after the second transaction (T2) has already accessed the uncommitted data—thus violating the isolation property of transactions.
- To illustrate that possibility, use the same transactions described during the lost updates discussion.
- T1 has two atomic parts, one of which is the update of the inventory; the other possible part is the update of the invoice total (not shown).
- T1 is forced to roll back due to an error during the updating of the invoice's total; it rolls back all the way, undoing the inventory update as well.
- This time the T1 transaction is rolled back to eliminate the addition of the 100 units. (See Table 10.5.) Because T2 subtracts 30 from the original 35 units, the correct answer should be 5.

TABLE 10.5

TRANSACTIONS CREATING AN UNCOMMITTED DATA PROBLEM

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100 (Rolled back)
T2: Sell 30 units	PROD_QOH = PROD_QOH – 30

Table 10.6 shows how the serial execution of these transactions yields the correct answer under normal circumstances.

TABLE 10.6

CORRECT EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	PROD_QOH = 35 – 30	
7	T2	Write PROD_QOH	5

Inconsistent Retrieval

- Inconsistent retrievals occur when a transaction accesses data before and after one or more other transactions finish working with such data.
- For example, an inconsistent retrieval would occur if transaction T1 calculated some summary (aggregate) function over a set of data while another transaction (T2) was updating the same data.
- The problem is that the transaction might read some data before it is changed and other data after it is changed, thereby yielding inconsistent results.

To illustrate the problem, assume the following conditions:

1. T1 calculates the total quantity on hand of the products stored in the PRODUCT table.

2. At the same time, T2 updates the quantity on hand (PROD_QOH) for two of the PRODUCT table's products.

The two transactions are shown in Table

TABLE 10.8

RETRIEVAL DURING UPDATE

TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = 1546-QQ2
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH – 10 WHERE PROD_CODE = 1558-QW1
	COMMIT;

- While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but meant to add the 10 units to product 1546-QQ2's PROD_QOH.
- To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH.
- The initial and final PROD_QOH values are reflected in Table (Only a
- few PROD_CODE values are shown for the PRODUCT table. To illustrate the point, the sum for the PROD_QOH values is shown for these few products.)

TRANSACTION RESULTS: DATA ENTRY CORRECTION

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	(15 + 10) → 25
1558-QW1	23	(23 – 10) → 13
2232-QTY	8	8
2232-QWE	6	6
Total	**92**	**92**

Table 10.7 shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

TABLE 10.7

AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	PROD_QOH = 135 – 30	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

- Although the final results shown in Table 10.9 are correct after the adjustment, Table 10.10 demonstrates that inconsistent retrievals are possible during the transaction execution, making the result of T1's execution incorrect.
- The "After" summation shown in Table 10.10 reflects that the value of 25 for product 1546-QQ2 was read after the WRITE statement was completed. Therefore, the "After" total is 40 + 25 = 65. The "Before" total reflects that the value of 23 for product 1558-QW1 was read before the next WRITE statement wa completed to reflect the corrected update of 13. Therefore, the "Before" total is 65 + 23 = 88.
- The computed answer of 102 is obviously wrong because you know from Table 10.9 that the correct answer is 92.
- Unless the DBMS exercises concurrency control, a multiuser database environment can create havoc within the information system.

- Although the final results shown in Table 10.9 are correct after the adjustment, Table10.10 demonstrates that inconsistent retrievals are possible during the transaction execution, making the result of T1's execution incorrect.
- The "After" summation shown in Table 10.10 reflects that the value of 25 for product 1546-QQ2 was read after the WRITE statement was completed. Therefore, the "After" total is 40 + 25 = 65. The "Before" total reflects that the value of 23 for product 1558-QW1 was read before the next WRITE statement was completed to reflect the corrected update of 13. Therefore, the "Before" total is 65 + 23 = 88.
- The computed answer of 102 is obviously wrong because you know from Table 10.9 that the correct answer is 92. Unless the DBMS exercises concurrency control, a multiuser database environment can create havoc within the information system.

TABLE 10.10

INCONSISTENT RETRIEVALS

TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 – 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

Concurrent execution of Transactions

- Transaction processing system usually allow multiple transaction to run concurrently.
- Allowing multiple transaction to run concurrently and allowing multiple transaction to update data concurrently causes several complications with consistency of data.
- Ensuring consistency with concurrency require an extra work.

Concurrent execution of Transactions

- Two reasons to allow concurrency are:-
  - Improve throughput and resource utilization:-(Throughput – Number of transactions that can be executed in a given amount of time.)
  - Reduced waiting time.

- There may be a mix of transactions running on a system, some short and some long.
- If transactions are run serially, a short transaction may have to wait for a preceding long transaction to complete, which can lead to unpredictable delays in running a transaction.
- But concurrent execution reduces the unpredictable delays in running transactions.

TYPES OF SCHEDULE

- 1. Serial Schedule
- 2. Non-serial Schedule
- 3. Serializable schedule

1. Serial Schedule

- The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction.
- In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.
- For example: Suppose there are two transactions T1 and T2 which have some operations.

- Execute all the operations of T1 which was followed by all the operations of T2.
- Execute all the operations of T2 which was followed by all the operations of T1.
- In the given (a) figure, Schedule A shows the serial schedule where T1 followed by T2.
- In the given (b) figure, Schedule B shows the serial schedule where T2 followed by T1.

- If it has no interleaving of operations, then there are the following two possible outcomes:

(a)

Time

T_1	T_2

read(A);
A := A − N;
write(A);
read(B);
B := B + N;
write(B);	

	read(A);
	A := A + M;
	write(A);

Schedule A

(b)

Time

T_1	T_2

	read(A);
	A := A + M;
	write(A);

read(A);
A:= A - N;
write(A);
read(B);
B:= B +N;
write(B);	

Schedule B

2. Non-serial Schedule/ **Concurrent Execution**

- If interleaving of operations is allowed, then there will be non-serial schedule.
- It contains many possible orders in which the system can execute the individual operations of the transactions.
- In the given figure (c) and (d), Schedule C and Schedule D are the non-serial schedules. It has interleaving of operations.

Non-serial Schedule

(c)

T_1	T_2
read(A); A := A − N;	
	read(A); A := A + M;
write(A); read(B);	
	write(A);
B := B + N; write(B);	

Schedule C

(d)

T_1	T_2

read(A);
A := A − N;
write(A);	

	read(A);
	A := A + M;
	write(A);

read(B);
B := B + N;
write(B);	

Schedule D

Problems with Concurrent Execution

- In a database transaction, the two main operations are **READ** and **WRITE** operations. So, there is a need to manage these two operations in the concurrent execution of the transactions.
- following problems occur with the Concurrent Execution of the operations:
- **Problem 1: Lost Update Problems (W - W Conflict)**
- **Dirty Read Problems (W-R Conflict)**
- **Unrepeatable Read Problem (W-R Conflict)**/ *Inconsistent Retrievals Problem*

Problem 1: Lost Update Problems (W - W Conflict)

- The problem occurs *when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.*
- **Consider the below diagram where two transactions T_X and T_Y, are performed on the same account A where the balance of account A is $300.**

Time	T_x	T_y
t_1	READ (A)	—
t_2	A = A - 50	
t_3	—	READ (A)
t_4	—	A = A + 100
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

- At time t1, transaction T_X reads the value of account A, i.e., $300 (only read).
- At time t2, transaction T_X deducts $50 from account A that becomes $250 (only deducted and not updated/write).
- Alternately, at time t3, transaction T_Y reads the value of account A that will be $300 only because T_X didn't update the value yet.
- At time t4, transaction T_Y adds $100 to account A that becomes $400 (only added but not updated/write).
- At time t6, transaction T_X writes the value of account A that will be updated as $250 only, as T_Y didn't update the value yet.
- Similarly, at time t7, transaction T_Y writes the values of account A, so it will write as done at time t4 that will be $400. It means the value written by T_X is lost, i.e., $250 is lost.

Dirty Read Problems (W-R Conflict) / Uncommitted Data

- The dirty read problem occurs *when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction. There comes the Read-Write Conflict between both transactions.*

Time	T_x	T_y
t_1	READ (A)	—
t_2	A = A + 50	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

- At time t1, transaction T_X reads the value of account A, i.e., $300.
- At time t2, transaction T_X adds $50 to account A that becomes $350.
- At time t3, transaction T_X writes the updated value in account A, i.e., $350.
- Then at time t4, transaction T_Y reads account A that will be read as $350.
- Then at time t5, transaction T_X rollbacks due to server problem, and the value changes back to $300 (as initially).
- But the value for account A remains $350 for transaction T_Y as committed, which is the dirty read and therefore known as the Dirty Read Problem.

Unrepeatable Read Problem (W-R Conflict) / *Inconsistent Retrievals Problem*

- *Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.*

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	A = A + 100
t_4	—	WRITE (A)
t_5	READ (A)	—

UNREPEATABLE READ PROBLEM

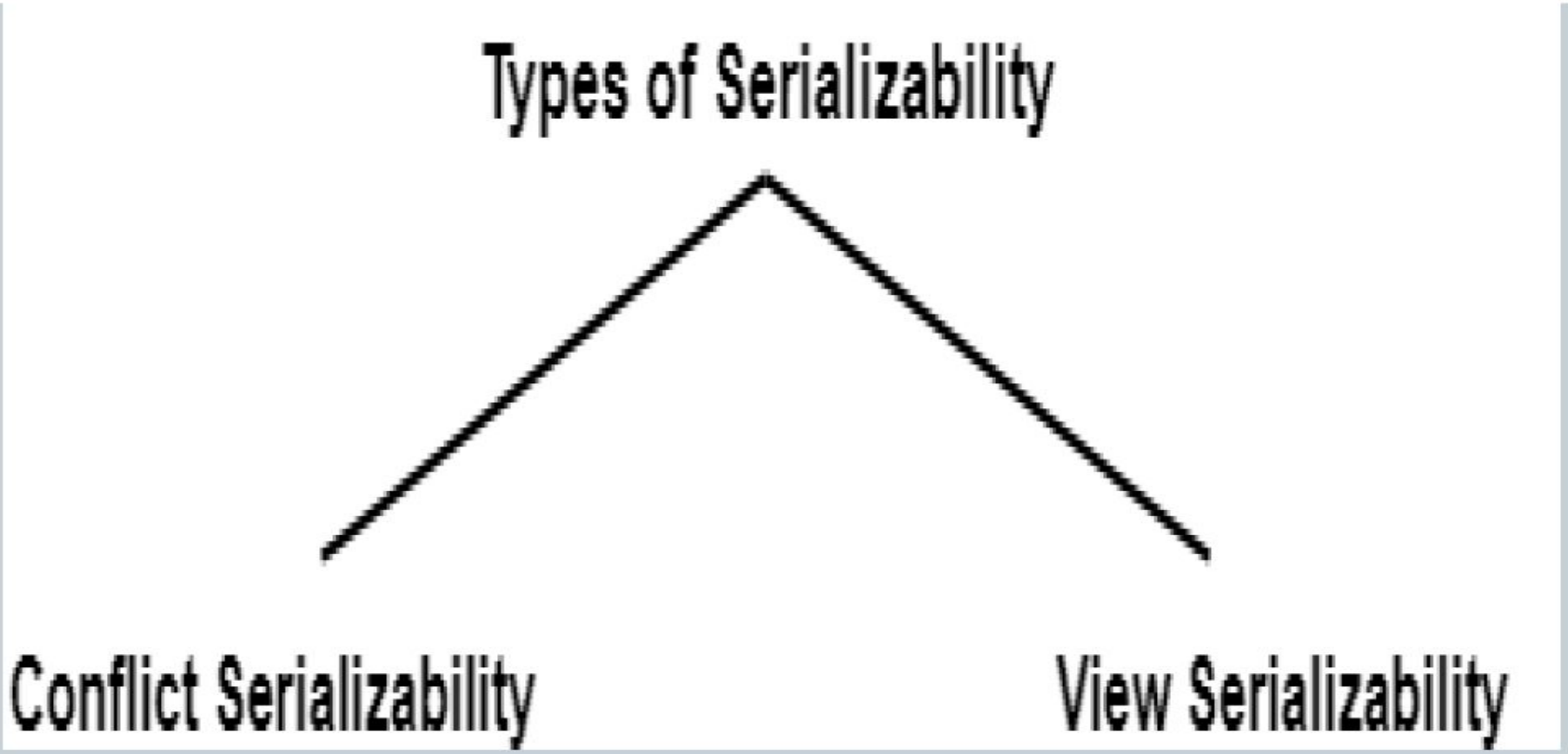
Serializability

- When multiple transactions run concurrently, then it may give rise to inconsistency of the database.
- **Serializability** is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database.
- If a given schedule of ‘n’ transactions is found to be equivalent to some serial schedule of ‘n’ transactions, then it is called as a **serializable schedule**.

Difference between Serial Schedules and Serializable Schedule

- The only difference between serial schedules and serializable schedules is that-
- In serial schedules, only one transaction is allowed to execute at a time i.e. no concurrency is allowed.
- Whereas in serializable schedules, multiple transactions can execute simultaneously i.e. concurrency is allowed.

Types of Serializability



The Scheduler

- A database transaction involves a series of database I/O operations that take the database from one consistent state to another
- **The scheduler** is a special DBMS process that ***establishes the order in which the operations are executed within concurrent transactions.***
- The scheduler interleaves the execution of database operations to ensure serializability and isolation of transactions.
- To determine the appropriate order, the scheduler bases its actions on concurrency control algorithms, such as **locking** or **time stamping** methods.

- It is important to understand that not all transactions are serializable.
- The DBMS determines what transactions are serializable and proceeds to interleave the execution of the transaction's operations.
- Generally, **transactions that are not serializable are executed on a first-come, first-served basis by the DBMS.**
- **The scheduler's main job is to create a serializable schedule of a transaction's operations,** in which the interleaved execution of the transactions (T1, T2, T3, etc.) yields the same results as if the transactions were executed in serial order (one after another).

- **The scheduler also makes sure that the computer's central processing unit (CPU) and storage systems are used efficiently.**
- If there were no way to schedule the execution of transactions, all of them would be executed on a first-come, first-served basis.
- The problem with that approach is that processing time is wasted when the CPU waits for a READ or WRITE operation to finish, thereby losing several CPU cycles.
- In short, **first-come, first-served scheduling tends to yield unacceptable response times within the multi user DBMS environment.**
- Therefore, some other scheduling method is needed to improve the efficiency of the overall system.

- **The scheduler facilitates data isolation to ensure that two transactions do not update the same data element at the same time.** Database operations might require READ and/or WRITE actions that produce conflicts.
- For example, Table 10.11 shows the possible conflict scenarios when two transactions, T1 and T2, are executed concurrently over the same data.
- Note that in Table 10.11, two operations are in conflict when they access the same data and at least one of them is a WRITE operation.

TABLE 10.11

READ/WRITE CONFLICT SCENARIOS: CONFLICTING DATABASE OPERATIONS MATRIX

	TRANSACTIONS		
	T1	T2	RESULT
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

Several methods have been proposed to schedule the execution of conflicting operations in concurrent transactions. These methods are classified as **locking**, and **time stamping**.

Concurrency Control with Locking Methods

- Locking methods are one of the most common techniques used in concurrency control because they facilitate the isolation of data items used in concurrently executing transactions.
- A lock guarantees exclusive use of a data item to a current transaction. In other words, transaction T2 does not have access to a data item that is currently being used by transaction T1.
- A transaction acquires a lock prior to data access; the lock is released (unlocked) when the transaction is completed so that another transaction can lock the data item for its exclusive use.
- ***The use of locks based on the assumption that conflict between transactions is likely*** is usually referred to as **pessimistic locking.**
- All lock information is handled by a **lock manager**, which is responsible for assigning and policing the locks used by the transactions.

Lock Granularity

- **Lock granularity indicates the level of lock use**.
- Locking can take place at the following levels: **database, table, page, row, or even field (attribute).**

Database Level :

- In a database-level lock, the entire database is locked, thus preventing

  the use of any tables in the database by transaction T2 while transaction T1 is being executed. This level of locking is good for batch processes, but it is unsuitable for multiuser DBMSs.

Figure 10.3 illustrates the database-level lock; because of it, **transactions T1 and T2 cannot access the same database concurrently even when they use different tables.**

FIGURE 10.3 DATABASE-LEVEL LOCKING SEQUENCE

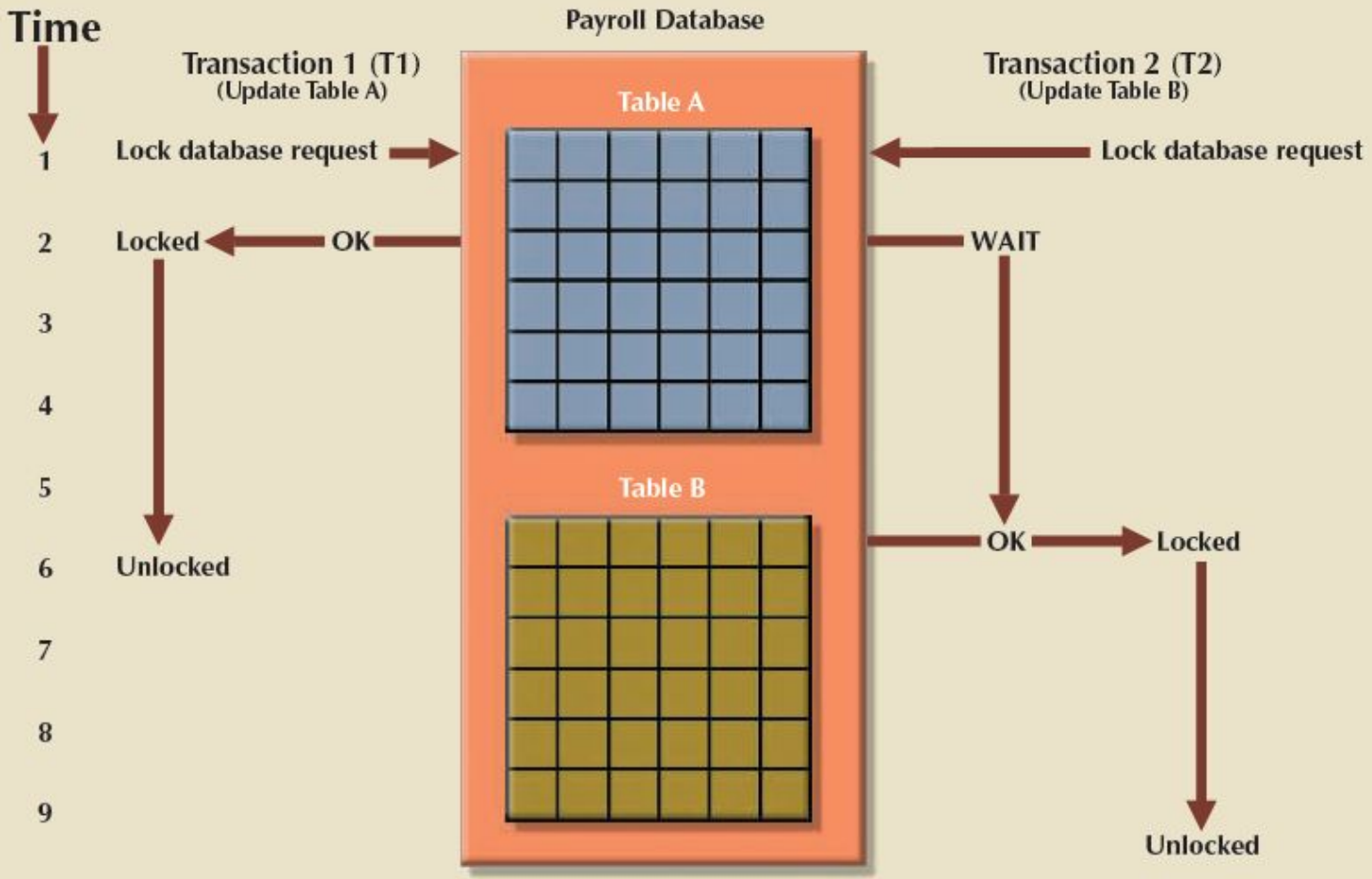
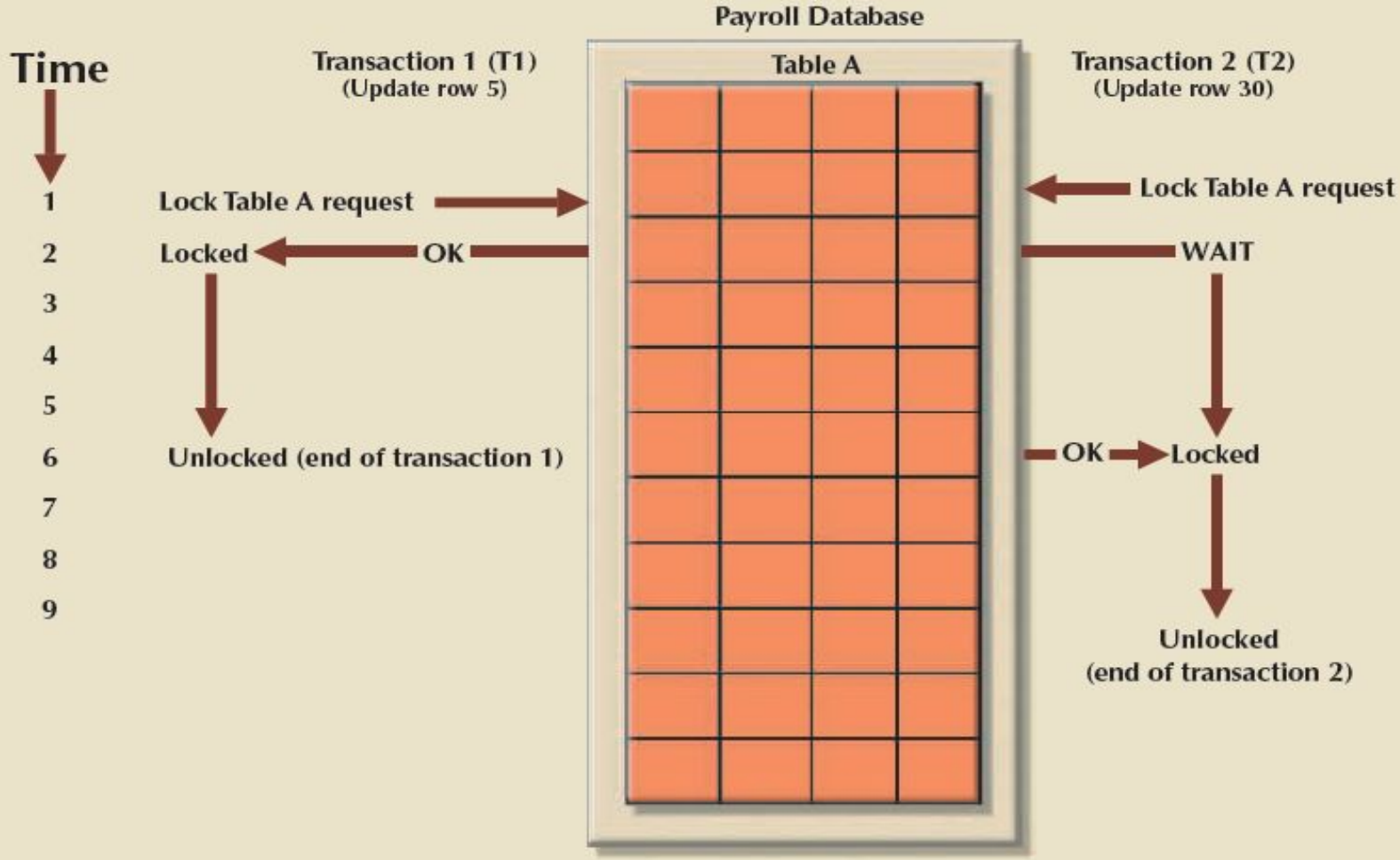


Table Level:

- In a table-level lock, **the entire table is locked, preventing access to any row by transaction T2 while transaction T1 is using the table.**
- If a transaction requires access to several tables, each table may be locked. However, two transactions can access the same database as long as they access different tables.
- Table-level locks, while less restrictive than database-level locks, cause traffic jams when many transactions are waiting to access the same table. Such a condition is especially irksome if the lock forces a delay when different transactions require access to different parts of the same table—that is, when the transactions would not interfere with each other.
- Consequently, table-level locks are **not suitable for multiuser DBMSs.**
- Figure 10.4 illustrates the effect of a table-level lock. Note that transactions T1 and T2 cannot access the same table even when they try to use different rows; T2 must wait until T1 unlocks the table.

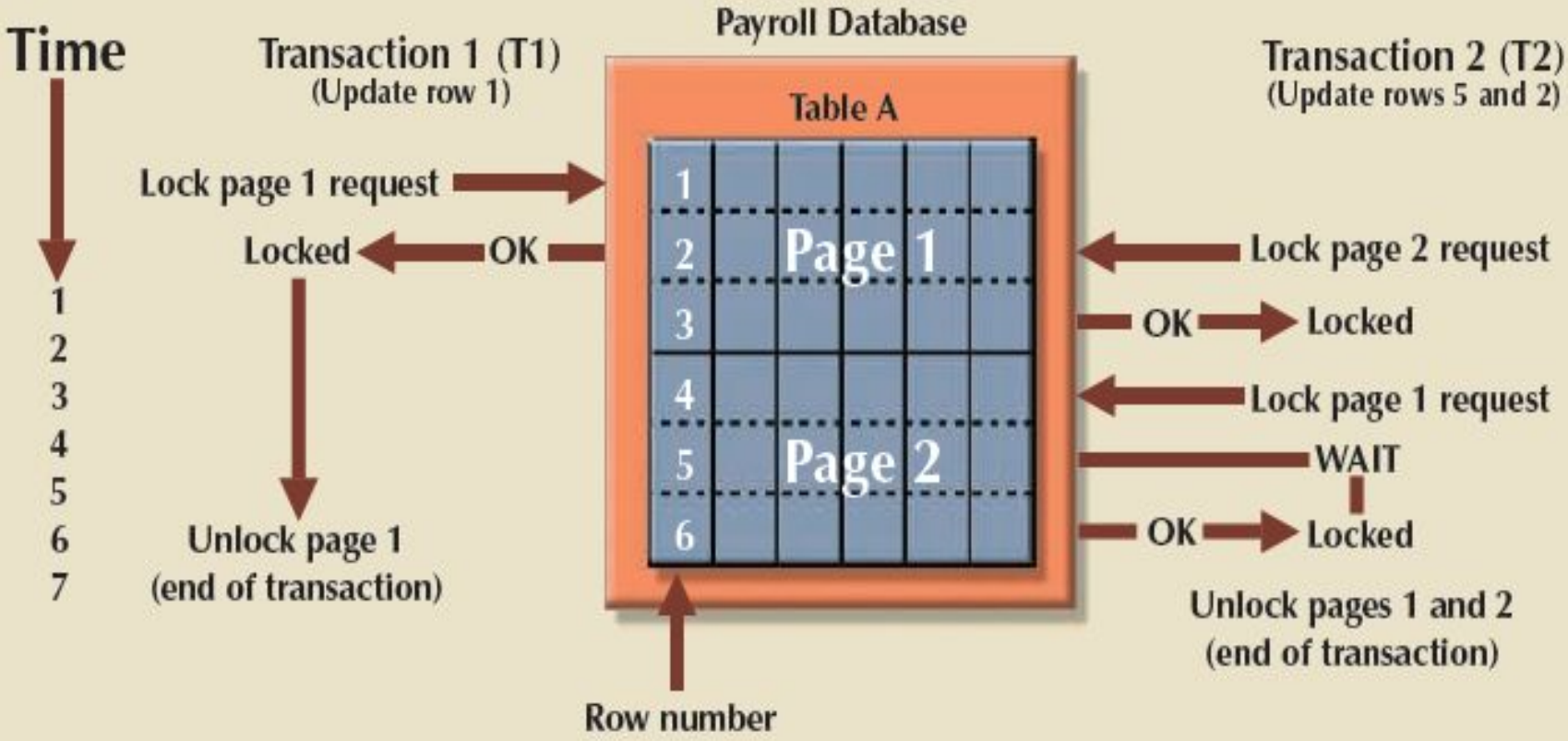
FIGURE 10.4 AN EXAMPLE OF A TABLE-LEVEL LOCK



Page Level:

- In a page-level lock, the DBMS locks an entire diskpage.
- A diskpage, or page, is the equivalent of a disk block, which can be described as a directly addressable section of a disk.
- A page has a fixed size, such as 4K, 8K, or 16K. For example, if you want to write only 73 bytes to a 4K page, the entire 4K page must be read from disk, updated in memory, and written back to disk.
- **A table can span several pages, and a page can contain several rows of one or more tables.**
- **Page-level locks are currently the most frequently used locking method for multiuser DBMSs**.
- An example of a page-level lock is shown in Figure 10.5. Note that T1 and T2 access the same table while locking different diskpages.
- If T2 requires the use of a row located on a page that is locked by T1, T2 must wait until T1 unlocks the page.

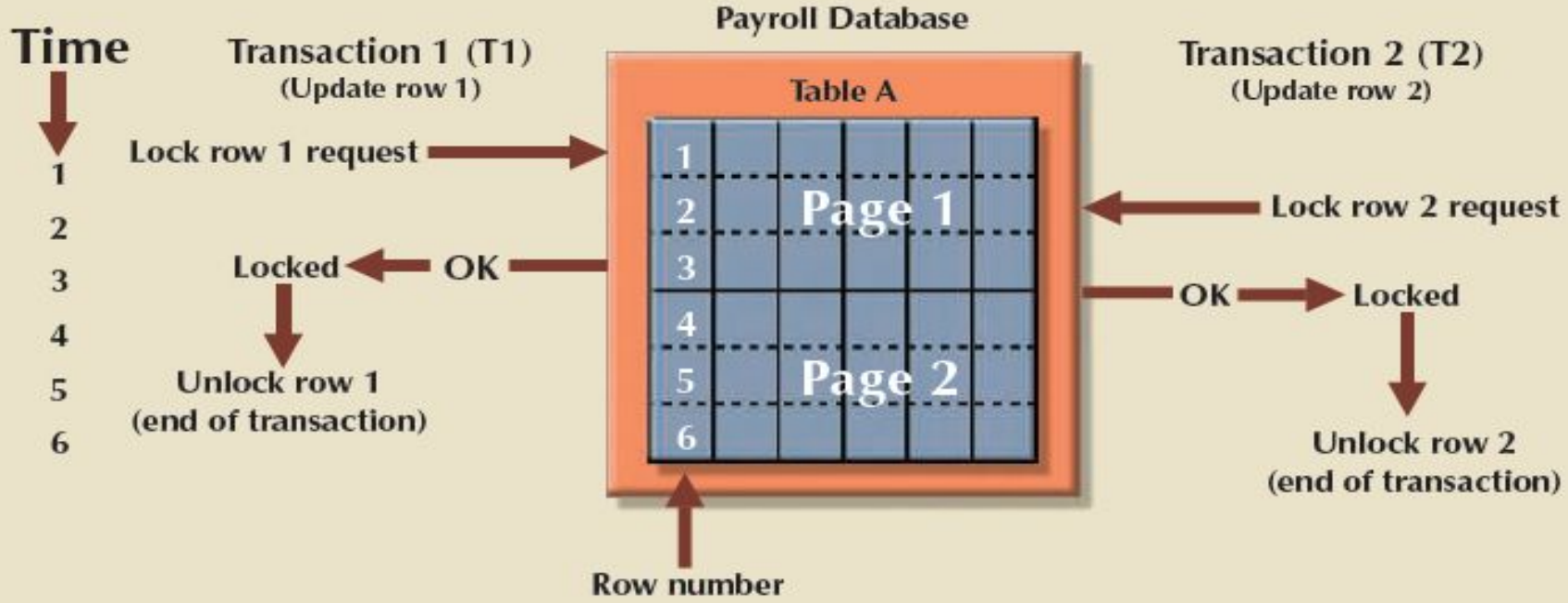
FIGURE 10.5 AN EXAMPLE OF A PAGE-LEVEL LOCK



Row Level:

- A row-level lock is much less restrictive than the locks discussed earlier.
- **The DBMS allows concurrent transactions to access different rows of the same table even when the rows are located on the same page.**
- Although the row-level locking approach improves the availability of data, its management requires high overhead because a lock exists for each row in a table of the database involved in a conflicting transaction.
- Modern DBMSs automatically escalate a lock from a row level to a page level when the application session requests multiple locks on the same page.
- Figure 10.6 illustrates the use of a row-level lock.
-

FIGURE 10.6 AN EXAMPLE OF A ROW-LEVEL LOCK



Note in Figure 10.6 that both transactions can execute concurrently, even when the requested rows are on the same page. T2 must wait only if it requests the same row as T1.

Field Level:

- **The field-level lock allows concurrent transactions to access the same row as long as they require the use of different fields (attributes) within that row.**

- Although field-level locking clearly yields the most flexible multiuser data access, it is rarely implemented in a DBMS because it requires an extremely high level of computer overhead and because the row-level lock is much more useful in practice.